

Operation Manual

Note

First we use docker. Each service can run in 1 or multiple docker containers on multiple servers. All servers are running Ubuntu os. You can make an ssh connection to a server.

Each docker container for service mount three directory

1. `/home/ubuntu/containers/logs/<docker-container-name> -> /logs`: this contains log of the service
2. `/home/ubuntu/containers/data/<docker-container-name> -> /data`: if a service wants to store some data. It can store here
3. `/home/ubuntu/containers/share/app_env.sh -> /appEnv.sh`: this file will be executed before the system runs. You can add some environment variables to this file. It will be available to all docker containers.
4. `/home/ubuntu/containers/share/service/<docker-container-name>.sh -> /appEnv/<docker-container-name>.sh`: this file will be executed right after `/appEnv.sh` and before the system runs. You can write some logic here for example: create `application.yml`, copy file, setting file permissions.

On each server, we have a folder: `/home/ubuntu/utils`. This folder contains some scripts that are useful during operating the system.

Useful Commands

Docker command

All the services are deployed at docker container. You should check some docker commands.

For example:

1. `docker ps`: list all docker container that are running
2. `docker restart <service name>`: restart a docker
3. `docker run`: run a docker container

Send kafka message command

Almost all services are using kafka. So you also can trigger some jobs for operation by sending a kafka message to the right topic. To do that you can make ssh connection to the server that has the Kafka container (server 172.33.10.67). Then run the following command:

```
cd /home/ubuntu
```

```
utils/produce_kafka.sh <topic name>
```

Then you paste the kafka message and press enter. Note that after pressing enter you are still in the command. So you can send more message to that topic. To exit from command, you can press ctrl +c

Log filter command

For log files that are not compressed with gz. You can use the ‘grep’ command. This command also supports regular expressions and helps you easier to filter logs. Also it support wildcard if you want to filter multiple files.

For log files that are already compressed with gz. You can use the command ‘zgrep’. This command is very similar to gz.

Basic Network Checking

1. Telnet
2. Curl
3. ping

Market Data

Re-Init Market Data

1. You can [send kafka message](#) to one of the following topic ‘market-collector-lotte-prod-9-1’, ‘market-collector-lotte-prod-9-3’, ‘market-collector-lotte-prod-9-5’, ‘market-collector-lotte-prod-9-7’, ‘market-collector-lotte-prod-9-8’, ‘market-collector-lotte-prod-10-1’, ‘market-collector-lotte-prod-10-3’ with content:
{"messageType":"REQUEST","messageId":"messageId","uri":"uncontrolled-job:/api/v1/collector/downloadSymbol","data":{}}

2. Or you can ssh to server 172.33.10.67. Then run command: “docker exec –ti kafka bash”. This command let you go inside docker container’s terminal. Now you can execute by run command: “./initMarket/init.sh” like iimage below:

```
ubuntu@NHSV-PROD-KAFKA-6:~$  
ubuntu@NHSV-PROD-KAFKA-6:~$  
ubuntu@NHSV-PROD-KAFKA-6:~$ docker exec -it kafka /bin/bash  
root@NHSV-PROD-KAFKA-6:/home/kafka# ./initMarket/init.sh
```

Resend market status

To resend market status, you can [send kafka message](#) to topic ‘marketStatus’. To recorrect market status you will need to identify which time is the starting time of the session and select messages from that. Below are the message available at starting time:

8h45

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HNX","status":"ATO","type":"DERIVATIVES","time":"020000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HOSE","status":"CLOSED","type":"EQUITY","time":"020000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HNX","status":"CLOSED","type":"EQUITY","time":"020000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"UPCOM","status":"CLOSED","type":"EQUITY","time":"020000","support":"test"}}
```

9h

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HOSE","status":"ATO","type":"EQUITY","time":"020000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HNX","status":"LO","type":"EQUITY","time":"020000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"UPCOM","status":"LO","type":"EQUITY","time":"020000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HNX","status":"LO","type":"DERIVATIVES","time":"020000","support":"test"}}
```

9h15

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HOS E","status":"LO","type":"EQUITY","time":"021500","support":"test"}}
```

11h30

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HOS E","status":"INTERMISSION","type":"EQUITY","time":"043000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HNX","status":"INTERMISSION","type":"EQUITY","time":"043000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"UPCOM","status":"INTERMISSION","type":"EQUITY","time":"043000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HNX","status":"INTERMISSION","type":"DERIVATIVES","time":"043000","support":"test"}}
```

13h

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HOS E","status":"LO","type":"EQUITY","time":"060000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HNX","status":"LO","type":"EQUITY","time":"060000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"UPCOM","status":"LO","type":"EQUITY","time":"060000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HNX","status":"LO","type":"DERIVATIVES","time":"060000","support":"test"}}
```

14h30

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HOS E","status":"ATC","type":"EQUITY","time":"073000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HNX","status":"ATC","type":"EQUITY","time":"073000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HNX","status":"ATC","type":"DERIVATIVES","time":"073000","support":"test"}}
```

14h45

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HNX","status":"CLOSED","type":"DERIVATIVES","time":"074500","support":"test"}}
```

15h

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HOSE","status":"CLOSED","type":"EQUITY","time":"080000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HNX","status":"CLOSED","type":"EQUITY","time":"080000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"HNX","status":"CLOSED","type":"DERIVATIVES","time":"080000","support":"test"}}
```

```
{"messageType":"REQUEST","messageId":"messageId","uri":"/update","data":{"market":"UPCOM","status":"CLOSED","type":"EQUITY","time":"080000","support":"test"}}
```

Restart servers

After restart all servers you need to do following:

1. Checking mysql is running in server 172.33.10.65 and 172.33.10.66 by command: “sudo systemctl status mysql”
2. Checking mongodb is running in server 172.33.10.66: “docker ps | grep mongo” and checking mongo container status
3. Checking redis is running in server 172.33.10.65: “sudo systemctl status redis”
4. Reseting kafka: ssh to server 172.33.10.67
 1. Run command /home/ubuntu/start-kafka.sh
 2. After this command finished. Wait for few seconds for kafka broker is up. You can check the log of kafka containers. And then send test message to topic “test”
 3. After the broker is up. Run command /home/ubuntu/setTopicPartition.sh to set partition number for special topics
5. You should restart all containers except minio on server 172.33.30.39
6. You can restart all containers in the following servers: 172.33.30.38, 172.33.30.40, 172.33.10.70, 172.33.10.71

Check Init Market Got Problem

Check which instance is running init market at 08:50

1. Ssh server 172.33.10.70
2. Run command: `/home/ubuntu/operatorScripts/find_instance_running_market_init.sh` to find which instance is running market init. If you see log with “masterextracted_**” or “master_engextracted” under “find in container” then the line starts with “find in container market-collector-lotte-prod-<a number>” is what we looking for. Please note the number in the end of matched line. Then goto step 4
3. If you don't find log what we need. Then SSH server 172.33.10.71 and repeat step 2. If you can find move to step 4. If you cannot find in both servers (there was problem) ,you should move to step 7.
4. Run command `/home/ubuntu/operatorScripts/find_issue_market_init.sh <number noted in step 2>`.
5. If commands shows logs with “fail to query” then there is problem on querying that symbol from HTS. Move to Step 7
6. If not, then there is no query info problem with HTS
7. Ssh server 172.33.10.67
8. Run command: “`docker exec -ti kafka bash`”
9. Run command: “`./initMarket/init9-2.sh`”
10. SSH server 172.33.10.70
11. Run command “`docker logs -tail 100 -f market-collector-lotte-prod-2`” and monitor the init process.
12. When init-process finishes it should have the log with “/refreshData”
13. If the process finishes with no errors, then you should restart related services:
 - a. Run command: “`docker restart -t 0 $(docker ps | grep coll | cut -d " " -f 1)`” on server 172.33.10.70 and 172.33.10.71
 - b. Run command “`docker restart -t 0 realtime-v2-prod`” on server 172.33.10.71